PROJECT SYNOPSIS

SecureReview AI

An AI-Powered Code Review and Security Auditing Platform
for Production-Readiness Assessment of AI-Generated Codebases

---

Submitted by	: [Your Name]
Roll / Registration No.	: [Roll Number]
Guided by	: [Guide Name], [Designation]
Department	: Computer Science & Engineering / Information Technology
Institution	: [College / University Name]
Academic Year	: 2025 – 2026
Degree Programme	: [B.Tech / B.E. / MCA / M.Tech — Final Year Project]

This synopsis is submitted in partial fulfilment of the requirements for the award of the degree in the above-mentioned programme.

Abstract

The widespread use of AI coding assistants such as GitHub Copilot and Cursor has drastically shortened the time needed to build working software, but it has also normalised the practice of shipping “vibe-coded” applications — code that runs, but was never reviewed for security, structure, or operational readiness. **SecureReview AI** is proposed as an AI-assisted code review and security auditing platform that ingests a codebase (via ZIP upload or a public GitHub repository link), runs it through a static analysis and rule-engine pipeline aligned with the OWASP Top 10, scores code quality and maintainability, and then uses a large language model (LLM) to translate raw findings into plain-English explanations and concrete before/after code fixes. The system aggregates these findings into a single, interpretable **Production Readiness Score**, giving developers — particularly students, hackathon teams, and early-stage startups — a fast, beginner-friendly answer to the question “is this code safe to deploy?”. The platform is built on a Java 17 / Spring Boot backend with a lightweight ML-based issue-prioritisation layer and an LLM-driven explanation layer, and is designed to be extensible to additional languages and CI/CD pipelines in future iterations.

Keywords: Static Application Security Testing (SAST), OWASP Top 10, Large Language Models, Code Quality, DevSecOps, Production Readiness, Spring Boot, AST Analysis.

1. Introduction

AI code generators let developers produce functional applications in hours rather than weeks. This speed, however, routinely comes at the cost of security and maintainability. AI-generated code is frequently insecure (SQL injection, XSS, authentication bypass, exposed secrets), poorly structured (high complexity, duplicated logic, no separation of concerns), and not production-ready (missing error handling, logging, or configuration management). Existing tooling addresses only part of the problem: static linters such as SonarQube or ESLint surface a large volume of technically accurate but often cryptic warnings, while AI chatbots can explain concepts in isolation but do not perform a structured audit of an entire, real codebase.

SecureReview AI is proposed to bridge this gap: a single platform that accepts real project code, performs static security and quality analysis, prioritises the resulting issues using a lightweight machine-learning model, and uses generative AI to explain each issue and its fix in accessible language — culminating in one composite Production Readiness Score.

2. Problem Statement

2.1 Background

Modern development culture rewards speed and iteration, and AI coding assistants amplify this further. The resulting risk surfaces along three dimensions:

- **Security** — SQL injection via string-concatenated queries, cross-site scripting, hardcoded API keys/passwords, and missing authentication or authorisation checks on sensitive endpoints.
- **Maintainability** — overly long methods, deep nesting, duplicated logic across files, and controllers that mix business logic and data-access responsibilities.
- **Operational readiness** — absent structured error handling, no logging or monitoring hooks, and configuration values (secrets, environment flags) hardcoded instead of externalised.

Teams with limited review bandwidth — students, solo founders, and early-stage startups in particular — often deploy such code directly to production, creating a realistic risk of data breaches, service outages, and accumulating technical debt.

2.2 Existing Solutions and Gaps

Category	Examples	Strengths	Limitations
SAST tools	SonarQube, Checkmarx, Snyk Code	Deep, rule-based analysis with broad language coverage	Complex setup; overwhelming warning volume; explanations skew technical; no unified “production readiness” concept
AI code assistants	GitHub Copilot, Cursor, Amazon Q / CodeWhisperer	Excellent at generating code quickly	Not built to audit existing codebases holistically; can introduce new vulnerabilities; no combined security + quality + readiness view
Educational platforms	OWASP guides, secure-coding tutorials	Strong conceptual knowledge base	Passive, reading-only content; not tied to the learner's own code; no automated feedback loop

Identified gap: there is no beginner-friendly, AI-driven tool that analyses a real project (ZIP/GitHub), combines security, quality and production-readiness signals into one view, explains issues in plain language, suggests concrete context-aware fixes, and reduces everything to a single “is this ready for production?” score. SecureReview AI is designed to fill precisely this gap.

3. Objectives

#	Objective	Description
1	**Automated code ingestion**	Accept code via ZIP upload or public GitHub repository URL; securely extract and stage it for analysis.
2	**OWASP-aligned static security analysis**	Detect Injection, Broken Access Control, Security Misconfiguration, Authentication Failures, Logging/Alerting Failures, and Cryptographic Failures in Java/Spring Boot code.
3	**Code quality & maintainability review**	Flag high cyclomatic complexity, duplication, poor naming, missing error handling, and weak test coverage signals.
4	**Production Readiness Scoring**	Compute a single 0–100 score from weighted security, quality and operational penalties, with a category-wise breakdown.
5	**AI-enhanced explanation and fix generation**	Use an LLM to explain each issue in plain English, show before/after code, and generate an executive summary.
6	**ML-based issue prioritisation**	Rank issues by severity, exploitability and impact on critical functionality using a lightweight classifier/heuristic model.
7	**Usable, filterable interface**	Provide a web UI to upload code, browse results, filter by severity/category, and copy suggested fixes.
8	**Educational value**	Link every issue to OWASP references and explain *why* a pattern is dangerous, reinforcing secure-coding habits over time.

4. Scope of the Project

4.1 In Scope (MVP)

- Primary language/framework: **Java with Spring Boot**; optional extension to JavaScript/Node.js and Python.
- Input via ZIP upload or a public GitHub repository URL.
- OWASP-aligned security vulnerability detection, code quality/maintainability checks, and a composite Production Readiness Score.
- AI-generated explanations and fix suggestions for every detected issue.
- Local deployment for demonstration, with optional cloud deployment (Render, Railway, or AWS).

4.2 Out of Scope (for MVP)

- Full OAuth integration for private GitHub repositories (proposed as future work).
- Deep CI/CD pipeline integration (GitHub Actions, Jenkins, etc.).
- Comprehensive support for every programming language.
- Real-time runtime application self-protection (RASP) or monitoring.
- Commercial-grade, multi-tenant scalability.

5. System Architecture

5.1 High-Level Architecture

The system follows a client–server architecture composed of five cooperating components:

Component	Responsibility
Frontend (Web UI)	React/Next.js (or Thymeleaf for a lean MVP) providing a ZIP-upload form, a GitHub-URL form, and a results dashboard (score, filterable issue list, code + fix viewer).
Backend (Spring Boot API)	Handles uploads and GitHub requests, manages secure extraction, orchestrates the analysis pipeline, calls the ML/LLM services, and stores/serves results.
Code Analysis Engine	File Scanner → Rule Engine → AST Analyzer (JavaParser / Eclipse JDT) → ML Prioritizer → LLM Explainer, run in sequence for every submitted project.
Data Storage	In-memory/file-based store for the MVP; PostgreSQL/MySQL with analysis_job and issue tables for the extended version.
External Services	GitHub REST API for repository downloads; an LLM API (cloud-hosted or a local model via Ollama/vLLM) for natural-language explanations.

5.2 Data Flow

- **Submit** — user uploads a ZIP (POST /api/analyze) or a GitHub URL (POST /api/analyze/github).
- **Validate & extract** — backend validates the input and extracts it into an isolated, size-limited temporary directory, guarding against Zip Slip.
- **Scan** — the file scanner selects relevant source and configuration files (*.java, pom.xml, application.properties, etc.).
- **Analyse** — the rule engine and AST analyzer collect raw issues (file, line, type, snippet); the ML model assigns a priority score to each.
- **Explain** — the LLM generates a plain-English description, a before/after fix, and an overall summary report.
- **Score** — the Production Readiness Score is computed from the weighted issue set.
- **Present** — results are persisted and exposed via GET /api/analysis/{jobId}; the frontend renders the dashboard.
- **Clean up** — temporary extracted files are deleted immediately after analysis or via a scheduled cleanup job.

5.3 Conceptual Module Diagram (textual)

```
[ Upload / GitHub URL ] → [ Secure Extractor ] → [ File Scanner ]
→ [ Rule Engine + AST Analyzer ] → [ ML Prioritizer ]
→ [ LLM Explainer ] → [ Scoring Engine ] → [ Results Store ] → [ Dashboard UI ]
```

Fig. 1 — Linear analysis pipeline from code submission to dashboard rendering.

6. Technologies and Tools

Layer	Technology	Purpose
Backend	Java 17+, Spring Boot 3.x, Maven/Gradle	Core API, orchestration, security
Static analysis	JavaParser / Eclipse JDT, custom rule engine	AST-level pattern detection
File handling	java.util.zip / Apache Commons Compress	Safe ZIP extraction
Frontend	React/Next.js or Thymeleaf + HTML/CSS/JS	Upload forms, results dashboard
ML (prioritisation)	Python (FastAPI/Flask) or Java (Smile/DL4J)	Ranks issues 0–1 by risk
LLM (explanation)	Cloud API (e.g. Claude/OpenAI) or local model via Ollama/vLLM	Plain-English descriptions and fixes
Database	PostgreSQL / MySQL (optional for MVP)	analysis_job, issue tables
DevOps	Docker, GitHub Actions (optional)	Containerised local/cloud deployment

7. Key Features (Detailed)

7.1 Code Ingestion

- **ZIP upload:** configurable max size (e.g. 25 MB); extension and MIME-type validation; extraction into an isolated temp directory with canonical-path checks to prevent Zip Slip.
- **GitHub URL:** parses owner/repo, calls GET /repos/{owner}/{repo}/zipball, and reuses the ZIP pipeline for consistency.
- **File filtering:** analyses source files (*.java, *.js, *.ts, *.py) and config files (pom.xml, build.gradle, package.json, .env); ignores binaries, images and generated docs to save time.

7.2 Security Analysis Rules (OWASP-Aligned)

OWASP Category	Example Detection Pattern
A05 — Injection	String-concatenated SQL ("...WHERE id='"+userId+"'"); shell/command injection via Runtime.exec(userInput).
A01 — Broken Access Control	Admin endpoints without @PreAuthorize/@RolesAllowed; direct object references with no ownership check.
A02 — Security Misconfiguration	Hardcoded secrets (password="...", api_key="..."); debug=true left enabled in production-like config.
A07 — Authentication Failures	Plain-text password storage (no BCrypt/SCrypt); fixed or non-expiring session identifiers.
A09 — Logging & Alerting Failures	Sensitive data (passwords, tokens, PII) written directly to application logs.
A04 — Cryptographic Failures	Use of weak hashes (MD5/SHA-1) for passwords; hardcoded encryption keys.

Each rule reports: file path, line number(s), the vulnerable snippet, category and severity.

7.3 Code Quality & Maintainability Checks

- **Complexity:** methods longer than a configurable threshold (e.g. 50 lines); nesting deeper than 4 levels.
- **Duplication:** similar blocks across files via text/AST similarity.
- **Naming & structure:** vague identifiers (data, temp, func1); controllers performing DB access or business logic instead of delegating to services.
- **Error handling:** empty catch blocks; broad catch(Exception) with no logging or recovery.
- **Test-coverage hints:** absence of src/test files; public methods with no corresponding test class.

7.4 Production Readiness Score

The composite score is computed as:

$$\text{Score} = 100 - (w_s \cdot S + w_q \cdot Q + w_o \cdot O)$$

Symbol	Meaning	Suggested Weight
S	Normalised security penalty (count & severity of security issues)	w_s = 0.60
Q	Quality penalty (complexity, duplication, naming, structure)	w_q = 0.25
O	Operational penalty (logging, config management, error handling)	w_o = 0.15

The dashboard presents the overall value, a category-wise breakdown, and the top 5 recommended fixes that would most improve the score.

7.5 AI-Powered Explanations & Fixes

For every detected issue the system generates: a plain-English description of the risk, a concrete before/after code fix, a one-line note on why the fix matters, and — at the project level — a 5–10 line executive summary covering the main risks, the recommended fix order, and an overall production readiness verdict.

Example — SQL Injection

```
Before:
String query = "SELECT * FROM users WHERE id = '" + userId + "'";

After:
String query = "SELECT * FROM users WHERE id = ?";
preparedStatement.setString(1, userId);
```

8. Feasibility Study

Dimension	Assessment
Technical	All core components (Spring Boot, JavaParser/Eclipse JDT, ZIP handling, REST APIs, LLM/ML APIs) are mature, well-documented, and freely available, making the MVP achievable within the proposed timeline.
Economic	Development relies on open-source libraries; the only recurring cost is optional LLM API usage, which can be minimised using free-tier or local models (Ollama) during development and demonstration.
Operational	The web-based interface requires no specialised training; developers already familiar with ZIP uploads and GitHub URLs can use the tool immediately.
Schedule	The scoped MVP (Java/Spring Boot only, ZIP + public GitHub input) is realistically deliverable within a 10–14 day focused sprint, or 6–8 weeks alongside coursework.

9. System Requirements

9.1 Hardware Requirements (Development)

- Processor: Intel i5 / AMD Ryzen 5 or higher
- RAM: 8 GB minimum (16 GB recommended if running a local LLM)
- Storage: 10 GB free space (for temp extraction, dependencies, and optional local models)
- Internet connection: required for GitHub API access and cloud LLM calls

9.2 Software Requirements

- OS: Windows 10/11, macOS, or Linux
- JDK 17 or later; Maven or Gradle
- Node.js 18+ (if using a React/Next.js frontend)
- Python 3.10+ (if the ML/LLM microservice is implemented separately)
- Docker (optional, for containerised deployment)
- PostgreSQL/MySQL (optional, for the extended persistence layer)

10. Implementation Plan (Timeline)

Proposed for a focused 10–14 day development window:

Days	Milestone
1–2	Project setup: Spring Boot skeleton, ZIP upload endpoint, secure extraction (Zip Slip protection), basic upload UI.
3–5	Static analysis engine: file scanner + 5–7 core security rules (SQL injection, hardcoded secrets, missing auth, plaintext passwords, sensitive logging).
6–7	Code quality rules (method length, nesting depth, basic duplication); Production Readiness Score calculation; results rendered on the dashboard.
8–9	GitHub integration: repository download via GitHub API, reuse of the existing extraction/analysis pipeline, testing against 2–3 sample public repos.
10–11	AI layer: LLM-based explanations and fixes, executive summary generation, heuristic/ML-based issue prioritisation.
12–13	UI polish (severity/category filters, code+fix viewer); testing against known-vulnerable projects (e.g. OWASP Juice Shop) and clean projects for comparison.
14	Documentation, final report, and demo preparation.

11. Testing Strategy

Test Type	Purpose
Unit testing	Validate individual rule detectors (e.g. SQL-injection regex/AST matcher) against known vulnerable and safe code snippets.
Integration testing	Verify the end-to-end pipeline: upload → extraction → analysis → scoring → API response.
Security regression testing	Run the tool against OWASP Juice Shop and similar intentionally vulnerable projects to confirm known issues are detected.
Negative / false-positive testing	Run against clean, well-structured projects to check the tool does not over-flag safe patterns.
Security testing of the tool itself	Validate ZIP-extraction hardening (Zip Slip), file-size/type limits, and that uploaded code is never executed.

12. Expected Outcomes

- A working web application that accepts code via ZIP or GitHub URL, analyses it for security and quality, and displays a Production Readiness Score with detailed, categorised issues.
- AI-generated, plain-English explanations and before/after fixes for every detected issue.
- A project report covering the problem statement, system design, implementation details, and results with screenshots.
- A live demonstration analysing a typical AI-generated MVP, identifying its critical vulnerabilities, and showing the score improve after applying the suggested fixes.
- A foundation that can be extended to more languages, CI/CD integration, and private-repository support.

13. Risk Analysis & Mitigation

Risk	Mitigation
False positives/negatives in static rules	Start with a small, well-tested rule set validated against known-vulnerable sample apps; expand incrementally.
LLM cost/latency for large codebases	Batch and cache LLM calls per unique issue pattern; fall back to a local model (Ollama) for demos.
Malicious uploaded code (zip bombs, Zip Slip, arbitrary write)	Enforce file-size/type limits, canonical-path validation on extraction, and never execute uploaded code.
Scope creep beyond the MVP timeline	Strictly limit the MVP to Java/Spring Boot and the defined rule set; treat other languages and CI/CD as future work.

14. Future Enhancements

- OAuth-based support for scanning private GitHub repositories.
- Integration with GitHub Actions/GitLab CI for automatic pull-request reviews.
- Additional language support: Node.js, Python, Go.
- A deeper ML model trained on real-world vulnerability datasets to predict exploit likelihood.
- An IDE plugin (VS Code/IntelliJ) for real-time, in-editor feedback.
- Team dashboards tracking security and quality trends over time, with optional per-developer metrics.

15. Conclusion

SecureReview AI addresses a concrete, present-day problem: the widening gap between how fast AI tools let developers generate code and how carefully that code is actually reviewed before deployment. By combining static analysis, a lightweight ML prioritisation layer, and generative-AI explanations, the platform offers clear and actionable insight, a holistic view of production readiness, and an educational feedback loop that helps developers write safer code over time. The project demonstrates applied skills in Java/Spring Boot backend engineering, AST-based static analysis, applied ML for ranking, and secure coding practice in its own implementation — making it well-suited as a final-year or capstone project with both academic depth and practical relevance.

16. References

1 OWASP Foundation, “OWASP Top 10:2025,” owasp.org/Top10/2025/.

2 GitHub Docs, “REST API endpoints for repository contents,” docs.github.com/en/rest/repos/contents.

3 GitHub Docs, “Downloading source code archives,” docs.github.com/en/repositories/working-with-files/using-files/downloading-source-code-archives.

4 Snyk, “Arbitrary File Write via Archive Extraction (Zip Slip),” security.snyk.io/vuln/SNYK-JAVA-ORGOWASP-3097709.

5 PVS-Studio, “V5338. OWASP. Possible Zip Slip vulnerability,” pvs-studio.com/en/docs/warnings/v5338/.

6 JavaParser Project Documentation, javaparser.org.

7 Eclipse JDT Core Documentation, eclipse.org/jdt/.

8 Various AI code-review and DevSecOps tooling surveys (2025–2026).